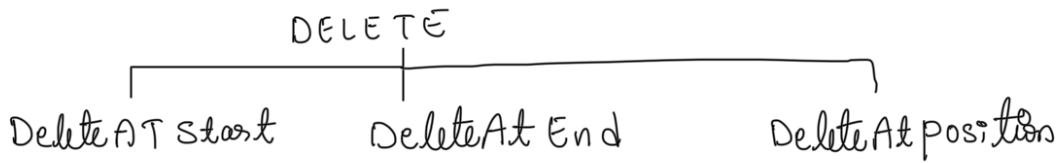
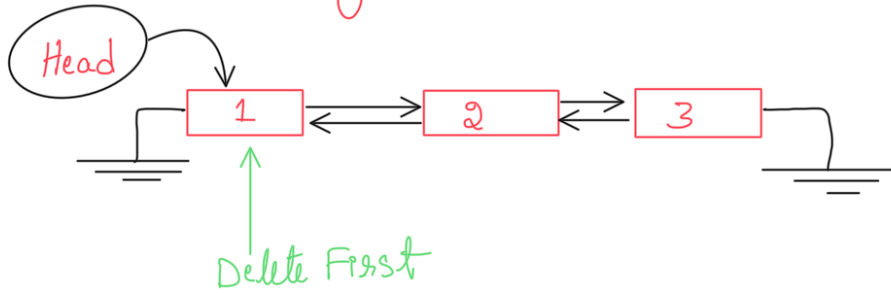


DELETE IN DOUBLY LINKED LIST

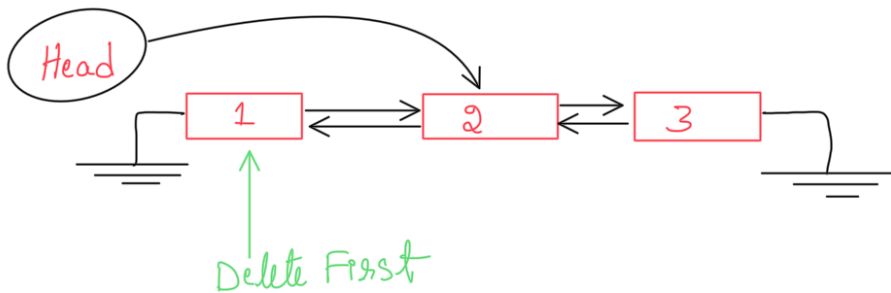
Day 42
21/08/2025



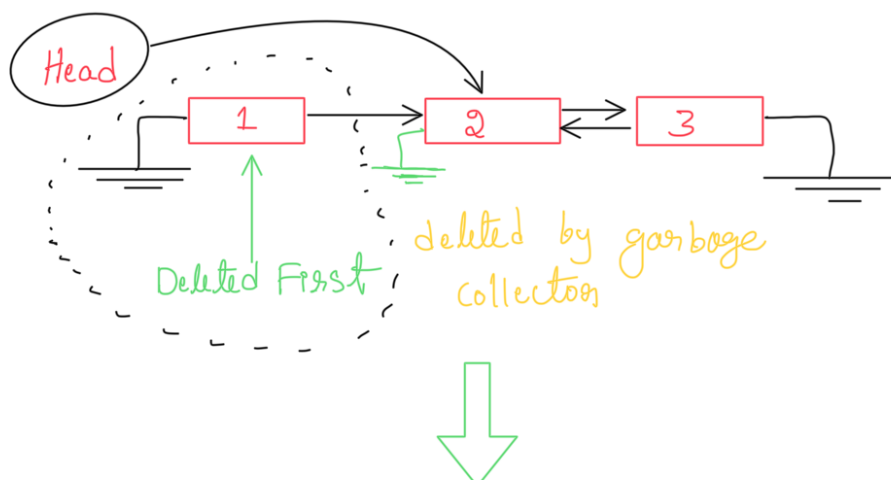
Delete At start (Deleting the HeadNode)

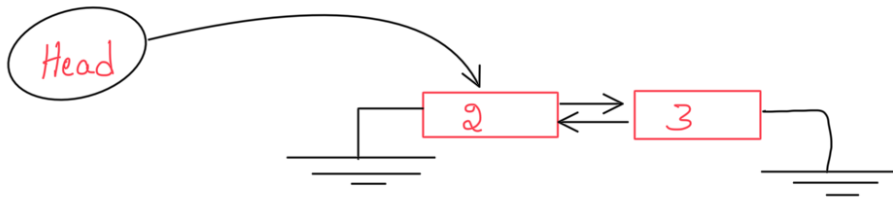


check if (Head.next != null)
shift Head to Head.next



Now Head is updated
and now also update Head.prev = null





First Node HeadNode is Successfully Deleted

Step 1
Shifting Head

Step 2
Head.prev = null

Corner case ①

if List Have Single Node

then only perform Step 1
Head = Head.next OR Head = Null

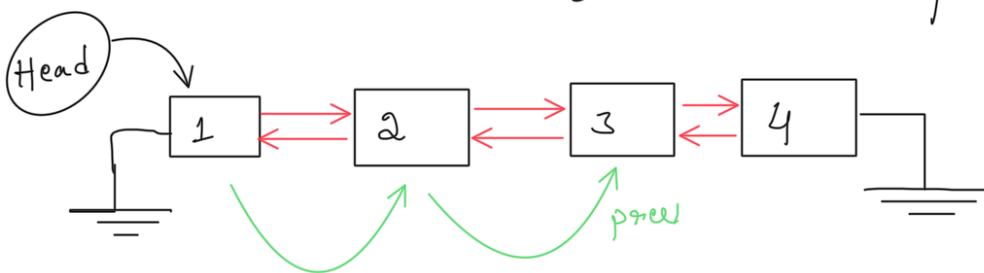
Corner case ②

if Head is Null
List is Empty So Just return

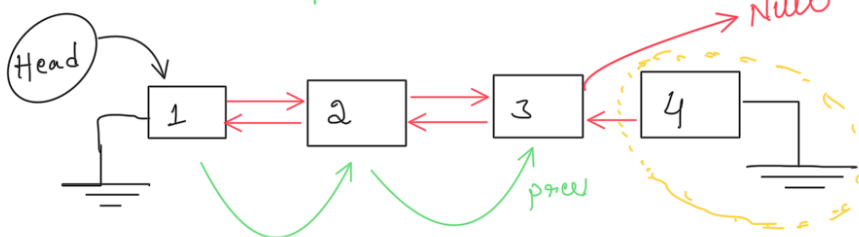
Single Node, Multinode, Empty List → Cases while deletion

DELETE AT THE END (Handling Single, Multi, Empty)

Reach the last second node & update Links



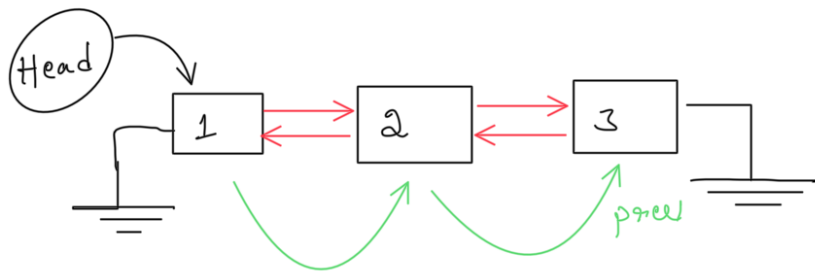
prev.next = null



odd last node
deleted By
garbage collector

prev.next = null

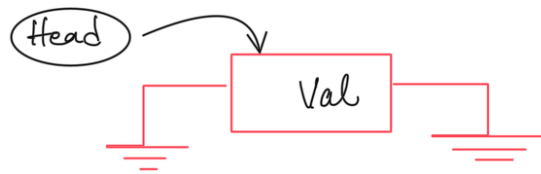




$prev.next = null$

Corner case (1)

The List has only single node

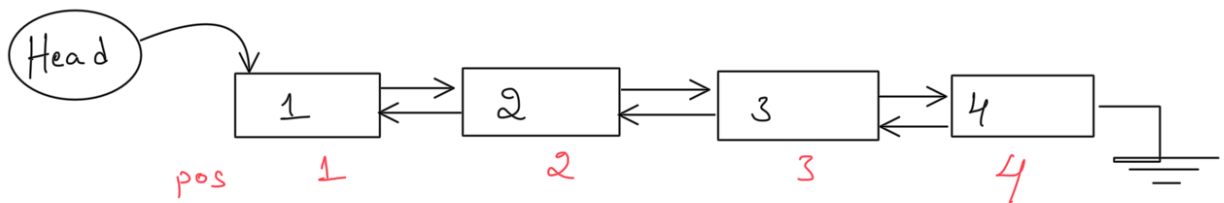


then Just assign $Head = null$

Corner case (2)

If $Head = null$ which means list is Empty So we just return

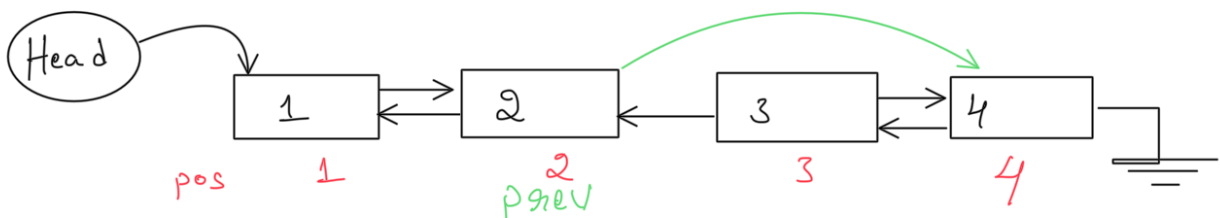
DELETE AT THE position



delete AT position 3

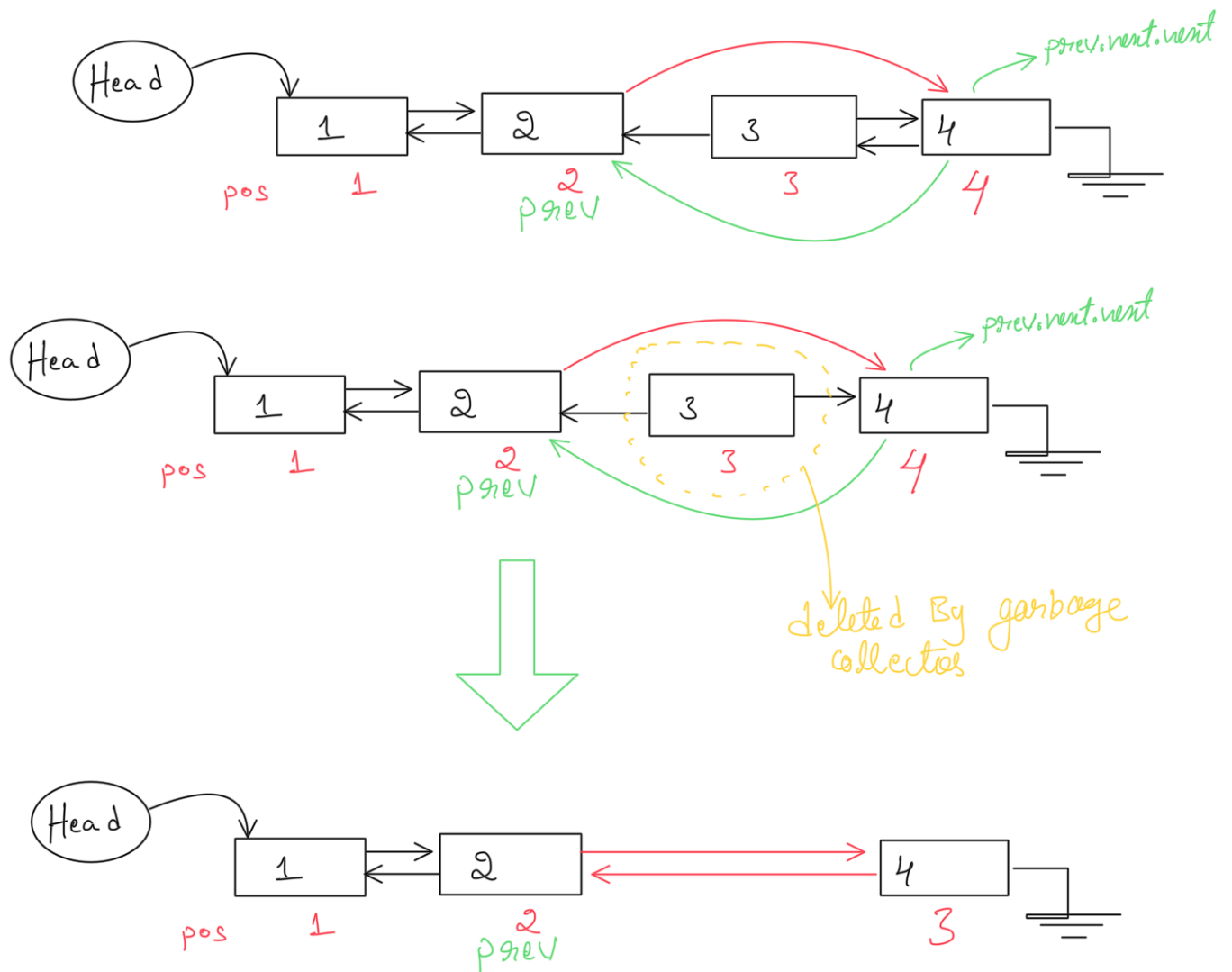
step 1 travel till prev node of delete position

$prev.next = prev.next.next;$



step 2

$$\text{prev.next.prev} = \text{prev}$$



step 1 $\text{delNode.next} = \text{delNode.next.next}$

step 2 $\text{delNode.next.prev} = \text{delNode}$

Corner Cases

Case (1) Empty list $\rightarrow$ return

Case (2) Single Node $\rightarrow$ if $(\text{pos} > 1) \rightarrow$ return

Case (3) $\text{pos} == 1$ deleteFront() $\rightarrow$ return

Case (4) $\text{pos} == \text{size}$ deleteEnd() $\rightarrow$ return

NOTE : DONT forget to update the size of linked list at each successful deletion (size--)

code

```
200
201 // Delete Operations
202
203 // Delete at front
204 public void deleteAtFront(){
205     if( this.head == null ) {
206         System.out.println(x:"The List IS Empty Can't Delete: ");
207         return;
208     }
209
210     if(this.head.next == null){
211
212         this.head =null;
213         this.size--; //Decrement Size ; Size updation
214         return;
215     }
216
217     this.head = head.next;
218     this.head.prev = null;
219     this.size--; // size updation
220
221 }
222
```

→ Corner Cases
Handled

```
222
223 // Delete at End
224 public void deleteAtEnd(){
225
226     if( this.head == null ) {
227         System.out.println(x:"The List IS Empty Can't Delete: ");
228         return;
229     }
230
231     if(this.head.next == null){
232         this.head =null;
233         this.size--; //Decrement Size ; Size updation
234         return;
235     }
236
237     Dnode lastNode = head ;
238
239     while ( lastNode .next != null) {
240         lastNode = lastNode .next;
241     }
242
243     lastNode .prev.next = null;
244     lastNode .next = null ; //optional Clean UP
245     this.size--; //Size updation
246 }
247
```

→ Corner Case

```
248 // DeleteAtPosition
249 public void deleteAtPosition(int position){
250     if( this.head == null ) {
251         System.out.println(x:"The List IS Empty Can't Delete: ");
252         return;
253     }
254
255     if (position == 1) {
256         deleteAtFront();
257         return;
258     }
259
260     if(position == this.size){
261         deleteAtEnd();
262         return;
263     }
264
```

→ Handling
Corner Cases

```

265         if(position <= 0 || position > this.size){
266             System.out.println(x:"Invalid Position ");
267             return;
268         }
269
270         int prevPosition = 1;
271         Dnode temp = head;
272
273         while (prevPosition < position - 1) {
274             temp = temp.next;
275             prevPosition++;
276         }
277
278         temp.next = temp.next.next;
279         temp.next.prev = temp;
280         this.size--; // size updation
281     }

```

The screenshot shows an IDE with the following components:

- Editor:** Displays `DoublyLinkedList.java` with test cases for deletion operations. A red box highlights the test cases from line 322 to 354.
- Output Window:** Shows the execution results of the test cases.


```

Inserted NewNode(15)
-----Nodes in the list are-----
null <==> 1 <==> 5 <==> 10 <==> 15 <==> null

Invalid position
-----Nodes in the list are-----
null <==> 1 <==> 5 <==> 10 <==> 15 <==>
reverse order
null <==> 15 <==> 10 <==> 5 <==> 1 <==> null

10 Key is found
99 Key is NOT found!

=== Testing Deletion Operations ===
Deleting at front...
-----Nodes in the list are-----
null <==> 5 <==> 10 <==> 15 <==> null

Deleting at end...
-----Nodes in the list are-----
null <==> 5 <==> 10 <==> null

Deleting at position 2...
-----Nodes in the list are-----
null <==> 5 <==> null

Deleting at invalid position...
Invalid Position
Deleting all elements one by one...
List is empty!
Deleting from empty list...
The List IS Empty Can't Delete:
The List IS Empty Can't Delete:
The List IS Empty Can't Delete:
=== End of Testing DoublyLinkedList ===
      
```
- Handwritten Annotations:**
 - A red box highlights the test cases in the code editor.
 - A red arrow points from the box to the text "Test Cases" written below the IDE.
 - The text "out put" is written in red above the output window.

Test Cases